

maketensor Code Documentation

FITS → HDF5 → scratch → online synthetic TNO injection → PyTorch tensors

1. Pipeline overview

The repository contains four Python files. Together they convert CLASSY difference-image FITS data into reusable HDF5 cutouts, *optionally* stage those shards onto scratch storage, inject synthetic TNOs at load time, and return PyTorch tensors for training.

1. FITS → HDF5	2. ARC → scratch	3. Synthetic injection	4. PyTorch loading
<code>fits2hdf5.py</code> Creates raw, unplanted HDF5 shards.	<code>stage2scratch.py</code> Copies raw shards to faster scratch storage.	<code>tno_injection.py</code> Plants synthetic moving TNOs using stored metadata.	<code>pytorch_hdf5_loader.py</code> Reads shards and returns (T, 3, H, W) tensors.

2. Quick-start workflow

```
# 1. Convert a visit/CCD into raw HDF5 shards
python fits2hdf5.py --visit 2022-08-01-AS1_July --ccd 15 \
    --num-samples 256 --samples-per-shard 64 --n-frames 8 --half-size 50

# 2. Copy shards from ARC home storage to scratch
python stage2scratch.py

# 3. Inspect one injected sample
python pytorch_hdf5_loader.py --data-dir /scratch/malvnair/shards_raw \
    --num-implants 1 --output-mode normal --save-fig

# 4. Benchmark DataLoader
python pytorch_hdf5_loader.py --data-dir /scratch/malvnair/shards_raw \
    --batch-size 16 --workers 0 2 4 8 --n-batches 50
```

The exact paths in the repository are currently specific to the NRC/ARC environment and may need to be changed by a future user.

fits2hdf5.py

Creates raw, unimplanted HDF5 cutout shards from CLASSY DIFFEXP FITS sequences

Main role: Read the original FITS sequence once, preserve the real background and metadata, extract repeated cutouts, and write compact HDF5 shards that can be reused during training.

Key inputs: visit list, DIFFEXP FITS files, TRIPPy PSF files, CCD number, frame count, cutout size.

Main output: one or more .h5 shard files containing science, variance, mask, coordinate transforms, per-frame metadata, PSF stamps, and subtraction-template metadata.

Command Line Arguments

Argument	Type/default	Meaning
--visit	str, required	Visit name, e.g. 2022-08-01-AS1_July
--ccd	int, default 15	CCD number
--num-samples	int, default 256	Total raw cutout sequences to generate
--samples-per-shard	int, default 64	Samples stored in each shard
--n-frames	int, default 8	Maximum science frames to load
--half-size	int, default 50	Half-width of square cutout; output is 2×half_size per side
--out-dir	str	Output directory for shards
--seed	int, default 0	random seed for cutout-center sampling

<code>--gzip</code>	flag	Enable gzip compression (used for downloading locally)
<code>--keep-template-order</code>	flag	Use visit-list order instead of sorting science frames by MJD

Function: `find_psf_file_from_dbimages`

Purpose: Locates the TRIPPy PSF file associated with one science image ID and CCD.

Signature: `find_psf_file_from_dbimages(image_id, ccd)`

Parameters:

- `image_id` (str or int): science image identifier.
- `ccd` (int): CCD index.

Returns:

- Path to the PSF FITS file if found.

Notes:

- Checks both `dbimages/<image_id>/ccd<ccd>/` and `dbimages/<image_id>/` for compatibility with two possible directory layouts.

Function: `build_wcs`

Purpose: Builds a 2D Astropy WCS object while cleaning a few header compatibility issues.

Signature: `build_wcs(header)`

Parameters:

- `header`: FITS header containing WCS keywords

Returns:

- `astropy.wcs.WCS`: 2D WCS object.

Notes:

- Copies RADECSYS into RADESYSa when needed and removes higher-order PV terms that are not used.

Function: `pixel_scale`

Purpose: Computes the mean WCS pixel scale in arcsec/pixel.

Signature: `pixel_scale(wcs_obj)`

Parameters:

- `wcs_obj` (`astropy.wcs.WCS`): image WCS.

Returns:

- float: mean pixel scale in arcsec/pixel.

Notes:

- Falls back to 0.185 arcsec/pixel if fails
-

Function: build_ref_to_frame_transform

Purpose: Approximates the local mapping from reference-frame crop coordinates to another exposure with a 2×3 affine transform.

Signature: `build_ref_to_frame_transform(wcs0, wcs_i, x0c, y0c)`

Parameters:

- `wcs0`: reference-frame WCS.
- `wcs_i`: target-frame WCS.
- `x0c, y0c` (float): reference crop origin.

Returns:

- `numpy.ndarray`, shape (2, 3): local affine transform.
-

Function: get_zeropoint

Purpose: Extracts the photometric zeropoint from PhotoCalib metadata or common FITS zeropoint keywords.

Signature: `get_zeropoint(hdul)`

Parameters:

- `hdul`: FITS HDUList.

Returns:

- float: photometric zeropoint; returns 0.0 if unavailable.

Notes:

- Primary relation: $zp = -2.5 \log_{10}(\text{calibrationMean}) + 31.4$.
-

Function: make_psf_stamp

Purpose: Renders a normalized, untrailed 64×64 PSF stamp from the stored TRIPPy PSF model.

Signature: `make_psf_stamp(psf_file, stamp_size=64)`

Parameters:

- `psf_file` (path-like): TRIPPy PSF model file
- `stamp_size` (int): output stamp size.

Returns:

- `numpy.ndarray` float32: normalized PSF image (sum is 1).
-

Function: load_visit

Purpose: Loads the science DIFFEXP sequence and all frame-level metadata for one visit/CCD.

Signature: `load_visit(visit, ccd, n_frames, sort_by_time=True)`

Parameters:

- `visit` (str): visit name.
- `ccd` (int): CCD index.
- `n_frames` (int): maximum number of science frames.
- `sort_by_time` (bool): sort by exposure midpoint if True.

Returns:

- dict of parallel lists: images, variance, masks, times, zeropoints, gains, WCS objects, pixel scales, PSF paths, seeing, filters, and WCS header strings.
-

Function: `load_templates`

Purpose: Loads metadata for the subtraction-template exposures used later to reproduce negative wells.

Signature: `load_templates(visit, ccd)`

Parameters:

- `visit` (str): visit name.
- `ccd` (int): CCD index.

Returns:

- dict: template image IDs, PSF files, times, exposure times, zeropoints, and pixel scales.
-

Function: `sample_center`

Purpose: Chooses a random reference-frame cutout center that remains safely inside every science frame.

Signature: `sample_center(rng, d, half_size)`

Parameters:

- `rng` (numpy Generator): random-number generator.
- `d` (dict): result from `load_visit`.
- `half_size` (int): cutout half-width.

Returns:

- tuple (x, y): safe reference-frame center.

Notes:

- Raises `RuntimeError` if a safe center cannot be found after 50 attempts.
-

Function: `cutout`

Purpose: Extracts a square image cutout around a rounded pixel center.

Signature: `cutout(image, x, y, half_size)`

Parameters:

- image (2D array): source image.
- x, y (float): center coordinates.
- half_size (int): half-width.

Returns:

- 2D copied array, or None if the cutout would leave the image boundary.

Function: write_shard

Purpose: Writes one HDF5 shard containing multiple cutout sequences and the metadata required for later synthetic injection.

Signature: `write_shard(out_path, d, tmpl, centers, half_size, run_attrs, compression=None)`

Parameters:

- out_path: output .h5 path.
- d: science visit metadata.
- tmpl: template metadata.
- centers: reference cutout centers.
- half_size: cutout half-size.
- run_attrs: file-level HDF5 attributes.
- compression: None or compression name such as gzip.

Returns:

- None; writes an HDF5 file.

Notes:

- science/variance/mask have shape (B, T, H, W).
- ref_to_frame_affine has shape (B, T, 2, 3).
- Stores per-frame PSF stamps and template metadata for later negative-well synthesis.

stage2scratch.py

Copies generated HDF5 shards from ARC home storage to scratch for faster local training access.

Configured source: /arc/home/malvnair/shards_raw

Configured destination: /scratch/malvnair/shards_raw

Function: main

Purpose: Creates the scratch destination directory, recursively copies all shard contents, then reports the total destination size.

Signature: `main()`

Parameters: None.

Returns:

- None.

Example usage:

```
python stage2scratch.py
```

Notes:

- Staging to scratch is not necessary to run the full program

tno_injection.py

Generates physically motivated synthetic TNO motion and renders sources into real cutout sequences at load time.

Current default orbit range: $r = 30\text{--}50$ AU, $e = 0\text{--}0.25$, $i = 0\text{--}35^\circ$; near-opposition approximation with $\alpha = \theta = 0$.

Current default magnitude range: 22–24 AB mag.

Function: PhotoCalib.get_zeropoint

Purpose: Reads the image photometric zeropoint

Signature: `PhotoCalib.get_zeropoint(hdul)`

Parameters:

- `hdul`: opened FITS HDUList.

Returns:

- float: zeropoint.

Function: PhotoCalib.mag_to_counts

Purpose: Converts apparent AB magnitude to expected total image counts.

Signature: `PhotoCalib.mag_to_counts(mag, zeropoint)`

Parameters:

- `mag` (float): AB magnitude
- `zeropoint` (float): image photometric zeropoint

Returns:

- float: expected counts

Notes:

- Uses $\text{counts} = 10^{((\text{zeropoint} - \text{mag})/2.5)}$.

Function: OrbitSampler.sample

Purpose: Draws one random synthetic TNO orbit parameter set.

Signature: `OrbitSampler.sample()`

Parameters: None

Returns:

- dict containing r, e, i_deg, i_rad, alpha_rad, and theta_rad.
-

Function: MotionModel.compute

Purpose: Converts sampled orbit parameters into apparent sky-plane motion near opposition.

Signature: `MotionModel.compute(orbit)`

Parameters:

- orbit (dict): output from OrbitSampler.sample()

Returns:

- dict with rate_ra, rate_dec, total rate, and position angle.

Notes:

- Uses an Earth angular-velocity scale of 148 arcsec/hour and opposition approximation.
-

Function: MagnitudeSampler.sample

Purpose: Draws a synthetic apparent magnitude uniformly from the configured range.

Signature: `MagnitudeSampler.sample()`

Parameters: None.

Returns:

- float: sampled magnitude
-

Function: build_line_psf

Purpose: Restores a TRIPPy PSF and converts it to a trailed line PSF for the TNO motion during one exposure.

Signature: `build_line_psf(psf_file, rate, rate_ra, rate_dec, exptime, pixel_scale)`

Parameters:

- psf_file: TRIPPy model path.
- rate, rate_ra, rate_dec: sky motion in arcsec/hour.
- exptime: exposure length in seconds.
- pixel_scale: arcsec/pixel.

Returns:

- TRIPPy modelPSF configured with line trailing
-

Function: make_psf_image

Purpose: Plants a trailed source model at a desired pixel coordinate and rescales it so its integrated flux equals the requested counts.

Signature: `make_psf_image(mpsf, shape, x, y, counts)`

Parameters:

- mpsf: trailed TRIPPy PSF.
- shape: output image shape.
- x, y: plant position.
- counts: target total counts.

Returns:

- 2D numpy array: noiseless source model.
-

Function: sample_visible_reference_position

Purpose: Samples a reference-frame position whose propagated track remains inside every science cutout.

Signature: `sample_visible_reference_position(rng, motion, affines, cent_times, cent_time0, pscales, H, W, pad_x, pad_y)`

Parameters:

- rng: random generator.
- motion: output of MotionModel.compute.
- affines: per-frame local transforms.
- cent_times: exposure center times.
- cent_time0: reference time.
- pscales: per-frame pixel scales.
- H, W: cutout size.
- pad_x, pad_y: edge margins.

Returns:

- tuple (u_ref, v_ref) or None
-

Function: inject_cutout_sequence

Purpose: Main public injection routine. Plants one or more synthetic TNOs into sequence, updates variance, optionally reproduces negative subtraction wells and Poisson noise, and returns training labels.

Signature: `inject_cutout_sequence(science, variance, mask, metadata, rng, num_implants=1, add_negative_wells=True, add_poisson_noise=True, mag_min=None, mag_max=None, edge_pad_frac=0.1, output_mode="normal")`

Parameters:

- science, variance, mask: arrays with shape (T, H, W).
- metadata: frame/template times, zeropoints, gains, pixel scales, PSF paths, and ref_to_frame_affine.
- rng: numpy Generator.
- num_implants: number of fake TNOs.
- add_negative_wells: reproduce template-subtraction wells.
- add_poisson_noise: add source photon noise.
- mag_min, mag_max: optional magnitude override.
- edge_pad_frac: position safety margin.

- `output_mode`: normal, mean, median, debug, or ground-truth.

Returns:

- science, variance, mask, labels.

Notes:

- Each implant is resampled until its full trajectory is visible in all frames.
- Variance is increased by the model Poisson contribution.
- The returned labels include orbital parameters, magnitude, sky rates, positive positions, negative-well positions, injected fluxes, and optional stack/track targets.

pytorch_hdf5_loader.py

Reads HDF5 shards, performs optional online injection, and returns tensors to PyTorch.

Main tensor shape: (B, T, 3, H, W)

Channel 0: science image

Channel 1: variance plane

Channel 2: raw CLASSY mask bitfield converted to float32

Function: `ShardDataset.__init__`

Purpose: Discovers all .h5 shards, reads each shard sample count, builds global index offsets, and configures online injection.

Signature: `ShardDataset(data_dir, base_seed=0, inject=True, num_implants=1, output_mode="normal")`

Parameters:

- `data_dir`: directory containing .h5 shards.
- `base_seed`: reproducibility seed.
- `inject`: enable/disable synthetic planting.
- `num_implants`: fake objects per sample.
- `output_mode`: injection output mode.

Returns:

- Initialized `torch.utils.data.Dataset` object

Function: `ShardDataset.__len__`

Purpose: Returns the total number of samples across every shard.

Signature: `len(dataset)`

Parameters: None.

Returns:

- int: total sample count
-

Function: ShardDataset.set_epoch

Purpose: Changes the epoch number so the same sample gets a new synthetic TNO each epoch.

Signature: `dataset.set_epoch(epoch)`

Parameters:

- epoch (int): current training epoch.

Returns:

- None.
-

Function: ShardDataset._get_file

Purpose: Opens the requested HDF5 shard and reopens files when needed after switching processes.

Signature: `dataset._get_file(shard_idx)`

Parameters:

- shard_idx (int): shard index.

Returns:

- h5py.File handle
-

Function: ShardDataset._get_meta

Purpose: Caches frame and template metadata that are constant for every sample in one shard.

Signature: `dataset._get_meta(shard_idx)`

Parameters:

- shard_idx (int): shard index.

Returns:

- dict of metadata arrays.
-

Function: ShardDataset._derive_rng

Purpose: Creates a deterministic random generator used for one sample and epoch.

Signature: `dataset._derive_rng(global_index)`

Parameters:

- global_index (int): sample index across all shards.

Returns:

- Numpy.random. generator

Notes:

- SeedSequence combines base_seed, epoch, and global sample index.
-

Function: `ShardDataset.__getitem__`

Purpose: Loads one sample from the correct shard, performs *optional* synthetic injection, stacks three channels, and returns a PyTorch tensor plus labels.

Signature: `dataset[global_index]`

Parameters:

- `global_index` (int): sample index across all shards.

Returns:

- dict with input: torch.Tensor of shape (T, 3, H, W), and labels: injection metadata/targets.
-

Function: `bench_one`

Purpose: Tests how quickly the DataLoader can load batches using a chosen number of workers.

Signature: `bench_one(dataset, batch_size, num_workers, n_batches)`

Parameters:

- `dataset`: ShardDataset.
- `batch_size`: samples per batch.
- `num_workers`: DataLoader worker processes.
- `n_batches`: number of steady-state batches to time.

Returns:

- prints samples/s and MB/s.
-

Function: `debug_injection`

Purpose: Runs a fake-only injection sanity check on sample 0, prints the injected frame positions, and saves a diagnostic image.

Signature: `debug_injection(data_dir, seed, num_implants)`

Parameters:

- `data_dir`: shard directory.
- `seed`: random seed.
- `num_implants`: number of synthetic objects.

Returns:

- debug PNG
-

Function: `save_output_figure`

Purpose: Loads one sample and saves a visualization for normal/debug/mean/median/ground-truth output modes.

Signature: `save_output_figure(data_dir, seed, num_implants, output_mode)`

Parameters:

- data_dir: shard directory.
- seed: random seed.
- num_implants: number of implants.
- output_mode: selected visualization mode.

Returns:

- saves sample_<output_mode>.png.

Command-line arguments

Argument	Type/default	Meaning
--data-dir	str, required	Directory containing .h5 shards
--batch-size	int, default 16	Samples per benchmark batch
--workers	list[int], default 0 2 4 8	Worker counts to benchmark
--n-batches	int, default 50	Steady-state batches per configuration
--seed	int, default 0	Controls random synthetic TNO generation.
--no-inject	flag	Measure raw loading only
--num-implants	int, default 1	Fake objects per sample
--output-mode	choice	normal, debug, mean, median, ground-truth
--save-fig	flag	Save one visualized sample and exit

--debug-injection	flag	Run injection sanity checks and save fake-only figure
-------------------	------	---

3. HDF5 shard layout

Dataset	Shape	dtype	Meaning
science	(B, T, H, W)	float32	Real difference-image cutouts
variance	(B, T, H, W)	float32	Variance planes
mask	(B, T, H, W)	uint16	Raw CLASSY mask bitfields
cutout_center	(B, 2)	float32	Reference-frame cutout centers
ref_pixel_origin	(B, 2)	float32	Reference crop pixel origins
ref_to_frame_affine	(B, T, 2, 3)	float32	Local reference-to-native coordinate transforms
frame/cent_time	(T,)	float64	Exposure midpoint times
frame/zp	(T,)	float32	Photometric zeropoints

frame/exptime	(T,)	float32	Exposure durations
frame/gain	(T,)	float32	Detector gain
frame/pixel_scale	(T,)	float32	Arcsec per pixel
frame/psf_file	(T,)	string	TRIPPy PSF model paths
frame/psf_stamp	(T, 64, 64)	float32	Normalized native untrailed PSF stamps
template/*	various	various	negative subtraction wells

4. Additional notes

- The repository currently assumes NRC ARC directory locations for visit lists, warps, dbimages, TRIPPy, and shard storage.
- The current tno_injection.py magnitude defaults are intentionally bright for testing (22–24 mag).
- The injection routine requires each synthetic TNO to remain inside every science frame. If no valid orbit/position combination is found after the configured retries, it raises a RuntimeError.